

Documento maestro — Diseño de Base de Datos (conceptual → lógico → normalización → optimización)

Este documento es tu **plantilla universal** para diseñar una base de datos (PostgreSQL) para sistemas administrativos tipo tienda (Arduino/electrónica), pero aplicable a casi cualquier CRUD empresarial.

Tú harás los diagramas (ER). Aquí te dejo el método completo para llegar a un modelo lógico oficial sin improvisar.

Filosofía (para que no se vuelva un monstruo)

- La base de datos (BD) no es solo “guardar cosas”: es el **cerebro de consistencia** del negocio.
 - Si el dominio está claro, el modelo sale fácil. Si el dominio es ambiguo, la BD se vuelve una trampa.
 - **Nada es obvio**: entidades, cardinalidades y reglas se preguntan y se documentan.
-

0) Antes de modelar: lo que debes tener a mano

Objetivo

Llegar a un modelo donde puedas defender por qué existe cada tabla y relación.

Entradas mínimas

1) **Narrativa del negocio** (levantamiento) 2) Lista de **casos de uso** (vender, comprar, pedidos, devoluciones) 3) Lista de **reglas de negocio** (no borrar, venta cerrada, etc.)

Precauciones

- Si modelas sin reglas, terminas “parchando” la BD con columnas random.
- Si cambian reglas tarde, el refactor cuesta caro.

Salida

- Documento base de contexto (lo que ya hiciste en tu proyecto)
-

1) Diseño conceptual (ER) — entidades, relaciones, cardinalidades

1.1 Objetivo

Construir el diagrama conceptual (ER) de forma defendible.

1.2 Pasos (método práctico)

Paso A — Extraer entidades (sustantivos)

De la narrativa, sacas “cosas” del negocio: - Producto, Categoría, SKU - Cliente, Proveedor - Venta, Ítem de Venta - Compra, Ítem de Compra - Pedido, Envío - Movimiento de Inventario - Usuario, Rol

Regla: si el negocio habla de ello como “algo” y tiene atributos → probablemente es entidad.

Paso B — Extraer relaciones (verbos)

- Cliente **realiza** Ventas
- Venta **contiene** Ítems
- Ítem **referencia** SKU
- Compra **ingresa** Inventario

Paso C — Poner cardinalidades

Define cuántos de cada lado: - Venta 1..N Ítems - Producto 1..N SKUs (si hay variantes) - Proveedor 1..N Compras

Paso D — Identificar entidades asociativas (muchos a muchos)

Cuando hay N..M: - Venta ↔ SKU (por medio de **SaleItem**) - Compra ↔ SKU (por medio de **PurchaseItem**)

Regla de oro: muchas a muchas **siempre** se vuelve tabla intermedia.

Paso E — Detectar atributos “multivalor” (trampa común)

Ejemplo de trampa: - `telefonos = "099...", 098..."`

Corrección: tabla `customer_phone` (1..N).

1.3 Checklist de calidad conceptual

✓ Cada entidad tiene identidad y propósito. ✓ Cada relación tiene cardinalidad. ✓ No hay “listas separadas por coma”. ✓ Procesos importantes tienen estado (pedido: pendiente→confirmado→...)

1.4 Precauciones (conceptual)

- Si mezclas **Pedido** con **Venta**, luego se rompe el negocio.
 - Pedido = intención
 - Venta = transacción
 - Si inventario es solo un número en producto, termina inconsistente.
-

1.5 Observaciones

En tiendas reales, el conceptual correcto suele girar alrededor de: - **Items** (sale_item, purchase_item) - **Movimientos** (inventory_movement) - **Estados** (orders)

2) Diseño lógico-relacional (primer modelo relacional)

2.1 Objetivo

Pasar del ER conceptual a tablas con PK/FK, listas de atributos y tipos.

2.2 Reglas de traducción ER → Relacional

2.2.1 Entidad → tabla

Entidad `Product` → tabla `product`.

2.2.2 Relación 1..N → FK en el lado N

• Category 1..N Product → `product.category_id`

2.2.3 Relación 1..1

Depende: - si es obligatoria y frecuente → misma tabla - si es opcional y pesada → tabla aparte

Ejemplo: - `product` + `product_specs` (opcional)

2.2.4 Relación N..M → tabla intermedia

`Sale` N..M `SKU` → `sale_item(sale_id, sku_id, qty, price)`

2.2.5 Atributo multivalor → tabla hija

Customer 1..N Phones → `customer_phone`

2.3 Convenciones (para que todo tenga orden)

Nombres

- Tablas en **snake_case** singular: `sale`, `sale_item`
- Campos FK: `customer_id`, `sale_id`
- Timestamps estándar:
- `created_at`
- `updated_at`

Keys (claves)

- PK: `id` (bigint o uuid)
- Natural keys (código de negocio):
- `sku_code` UNIQUE
- `username` UNIQUE

Soft delete (no borrar)

- `is_active` boolean o `deleted_at` timestamp

2.4 Modelo lógico típico para tienda (lista de tablas)

No es “tu única verdad”, es una referencia base para que tengas estructura.

Catálogo

- `category`
- `product`
- `sku`

Personas

- `customer`
- `supplier`

Operación

- `purchase`
- `purchase_item`
- `sale`
- `sale_item`

Inventario (pieza crítica)

- `inventory_movement`
- type: ENTRADA / SALIDA / AJUSTE / DEVOLUCION

Pedidos

- order
- order_item
- shipping

Seguridad

- user
- role
- user_role (si es N..M)

2.5 Precauciones (lógico inicial)

- No guardes stock como número editable sin control.
- Mejor: stock derivado de movimientos.
- No guardes total sin definición.
- Si lo guardas: define si es calculado o persistido.

3) Normalización (1FN → 2FN → 3FN)

3.1 Objetivo

Eliminar duplicación y dependencias incorrectas.

3.2 1FN (Primera forma normal)

Qué exige

- Cada campo debe contener un valor atómico (no listas)
- No repetir grupos de columnas

✓ Ejemplo correcto - customer_email (un valor)

✗ Ejemplo incorrecto - emails = "a@x.com, b@y.com"

3.3 2FN (Segunda forma normal)

Qué exige

- Si una tabla tiene PK compuesta, todos los atributos dependen de toda la PK.

Caso típico: tabla intermedia con PK compuesta.

✓ Ejemplo `sale_item(sale_id, sku_id, qty, unit_price)` - `qty` depende de `(sale_id, sku_id)`

✗ Incorrecto `sale_item(sale_id, sku_id, customer_name)` - `customer_name` depende de `sale_id`, no de `sku_id`

3.4 3FN (Tercera forma normal)

Qué exige

- No debe haber dependencias transitivas.

✗ Ejemplo clásico `sku(sku_id, category_name)` - `category_name` depende de `category_id`, no de `sku_id`

✓ Corrección - `sku` referencia a `product` - `product` referencia a `category`

3.5 Checklist de normalización

✓ No hay listas en campos. ✓ Los datos del cliente no se duplican en cada venta. ✓ Categorías no se repiten como texto por todas partes. ✓ Tablas intermedias no cargan datos que no les corresponden.

3.6 Precauciones

- 3FN es buen objetivo, pero la performance se resuelve con índices y queries, no con “romper normalización” por pánico.
-

4) Modelo lógico oficial (ya fino)

4.1 Objetivo

Dejar un modelo que puedas implementar en Postgres/Prisma sin dudas.

4.2 Qué define el “modelo oficial”

1) Tablas definitivas 2) PK/FK definitivas 3) Constraints definitivas 4) Índices iniciales 5) Estados/Enums

4.3 Reglas y criterios para tu modelo oficial

A) IDs y tipos

- Opción 1: `bigserial` (simple, rápido)

- Opción 2: `uuid` (mejor si habrá muchos servicios)

Regla: elige uno y sé consistente.

B) Enums vs tablas

- Enums para estados cerrados:
- `order_status`
- `movement_type`

C) Constraints que SIEMPRE debes tener

- UNIQUE en:
- `user.username`
- `sku.sku_code`
- CHECK en:
- cantidades ≥ 0
- precios ≥ 0

D) Acciones que NO haces con DELETE

- No borrar ventas
- No borrar compras
- No borrar movimientos

Solución: soft delete o flags.

4.4 Cómo modelar inventario de forma robusta

Enfoque recomendado: inventario por movimientos

`inventory_movement` - `id` - `sku_id` - `type` (ENTRADA/SALIDA/AJUSTE/DEVOLUCION) - `qty` - `reason` - `reference_type` (SALE/PURCHASE/RETURN/MANUAL) - `reference_id` - `created_by` - `created_at`

Stock actual = SUM(qty con signos según type)

Precaución

- Si guardas stock "final" en `sku.stock`, necesitas reglas estrictas y transacciones.

5) Reglas de diseño para NO volverte loco (heurísticas)

Estas son las "reglas de unir tablas" que a veces se dicen en clase, pero en versión profesional.

5.1 Cuándo separar una tabla

Separa cuando: - hay relación 1..N - hay datos opcionales y pesados - hay datos sensibles que requieren control - hay multivalor

Ejemplo: - `customer` y `customer_address` (1..N)

5.2 Cuándo mantener todo en una tabla

Mantén junto cuando: - es 1..1 obligatorio - siempre se consulta junto - la separación solo agrega joins innecesarios

Ejemplo: - `user` con campos básicos

5.3 Cuándo usar tabla intermedia (N..M)

Siempre que: - un A se relaciona con muchos B - y un B con muchos A

Ejemplo: - `user_role`

5.4 Evitar anti-patrones

- EAV (Entity-Attribute-Value) “tabla tipo Excel”
- Campos JSON para todo (sin control)
- Guardar listas en strings

6) Optimización (sin romper el modelo)

6.1 Objetivo

Hacer el sistema rápido y estable sin inventarte columnas raras.

6.2 Índices (la optimización #1)

Índices obligatorios

- todos los `*_id` usados como FK
- columnas de búsqueda frecuente:
 - `sku_code`
 - `product.name`

Índices compuestos (cuando aplica)

- filtros combinados:
 - `sale(created_at, status)`

Índices parciales (cuando aplica)

- solo activos:
 - `WHERE is_active = true`
-

6.3 Vistas (views) y materialized views

Views

- resumen de ventas
- resumen de stock

Materialized views (cuando el sistema crece)

- reportes pesados precalculados

Precaución: requiere estrategia de refresh.

6.4 Denormalización (solo si tienes razón real)

¿Cuándo se justifica?

- reportes lentos
- dashboards pesados
- demasiados joins repetidos

Ejemplos sanos

- guardar `sale.total` para no recalcular cada vez

Regla: si guardas total, debes definir: - cuándo se calcula - cuándo se bloquea (venta cerrada)

6.5 Transacciones y consistencia

Cuándo necesitas transacción sí o sí

- Cerrar venta + registrar movimientos
- Confirmar compra + registrar movimientos

Regla: operación crítica = atómica.

7) Triggers, funciones y reglas (con cautela)

7.1 Objetivo

Saber cuándo usarlos y cuándo NO.

7.2 Regla práctica

- Si puedes resolver en backend con transacciones → hazlo en backend.
- Usa triggers solo para:
 - auditoría
 - restricciones fuertes
 - automatismos muy estables

7.3 Precauciones

- Triggers invisibles hacen debugging difícil.
 - Si el trigger es complejo, documenta y testea.
-

8) Auditoría y trazabilidad (modo profesional)

Objetivo

Saber “quién hizo qué, cuándo y por qué”.

Opciones

1) `created_by`, `updated_by` en tablas críticas 2) Tabla `audit_log` (eventos)

Recomendación V1: - mínimo `created_by` en movimientos e items importantes.

9) Migraciones (cómo evolucionas sin romper)

Objetivo

Modificar BD sin caos.

Reglas

- Migraciones incrementales
 - Nunca editar migraciones ya aplicadas en prod
 - Mantener seeds para data base mínima
-

10) Checklist final (modelo listo para implementarse)

✓ Modelo conceptual (ER) con cardinalidades ✓ Modelo relacional inicial ✓ Normalización a 3FN ✓
Modelo lógico oficial (tablas finales) ✓ Constraints importantes (UNIQUE/CHECK) ✓ Índices iniciales ✓
Estrategia de inventario por movimientos ✓ Decisión: soft delete ✓ Plan de migraciones y seeds

Mini guía mental para ti (cuando dudes)

- Si es N..M → tabla intermedia.
 - Si hay lista dentro de un campo → tabla nueva.
 - Si algo no se debe borrar → soft delete.
 - Si un cambio debe ser histórico → tabla de movimientos/eventos.
 - Si duele performance → índices primero, denormalización después.
-

11) Ejemplo aplicado (ElectroMakers GYE) — así lo trabajamos paso a paso

Esta sección aterriza el método a **este proyecto real** (tienda Arduino/electrónica) para que tengas una receta repetible.

11.1 Punto de partida

Contexto del negocio (V1): - Venta presencial (mostrador) + ventas/pedidos por WhatsApp. - Catálogo por categorías. - Producto con **variantes vendibles** (SKU: original/compatible, marca, versión, voltaje). - Compras a proveedores. - Inventario cambia por compras, ventas, devoluciones y ajustes. - Ventas cerradas no se editan; correcciones vía devoluciones. - Pagos parciales permitidos.

✓ Resultado: esto define el **universo** de entidades y reglas.

11.2 Paso 1 — Elegir los “procesos núcleo” (lo que obliga al modelo)

Para este caso, el núcleo operativo fue: 1) Catálogo (categoría → producto → SKU) 2) Compras (compra → ítems) 3) Ventas (venta → ítems) 4) Pedidos/cotizaciones (pedido → ítems → estados) 5) Pagos (posibles pagos múltiples) 6) Inventario (movimientos) 7) Devoluciones (devolución → ítems) 8) Usuarios/roles (auditoría)

✓ Esta lista es lo que evita que te falten tablas “sin darte cuenta”.

11.3 Paso 2 — Entidades mínimas que sacamos

Catálogo

- **Categoría**
- **Producto**
- **SKU (VarianteProducto)** ← *esto es lo que realmente se vende*

Personas

- **Proveedor**
- **Cliente** (*opcional: puede existir venta como consumidor final*)

Operación

- **Compra + DetalleCompra**
- **Venta + DetalleVenta**
- **Pago**
- **Pedido + DetallePedido**
- **Envío** (*opcional, solo si el pedido requiere entrega*)
- **MovimientoInventario**
- **Devolución + DetalleDevolución**

Seguridad

- **Usuario**
- **Rol**

✓ Observación importante del caso: - Separar **Producto** y **SKU** fue la decisión correcta porque el negocio vende variantes (original/compatible, marca, versión, voltaje...).

11.4 Paso 3 — Cardinalidades clave (la columna vertebral)

Estas fueron las relaciones que definieron todo el diseño:

Catálogo

- Categoría **1** — **N** Producto
- Producto **1** — **N** SKU

Compras

- Proveedor **1** — **N** Compra
- Compra **1** — **N** DetalleCompra
- SKU **1** — **N** DetalleCompra
- Usuario **1** — **N** Compra (quién registra)

Ventas

- Cliente **0..1** — **N** Venta (cliente opcional)
- Venta **1** — **N** DetalleVenta

- SKU 1 — N DetalleVenta
- Venta 1 — N Pago (pagos parciales)
- Usuario 1 — N Venta (vendedor)

Pedidos (WhatsApp)

- Cliente 0..1 — N Pedido
- Pedido 1 — N DetallePedido
- SKU 1 — N DetallePedido
- Pedido 0..1 — 0..1 Envío (si es retiro no existe)
- Pedido 0..1 — 0..1 Venta (si se convierte)
- Usuario 1 — N Pedido (quién atendió)

Inventario

- SKU 1 — N MovimientoInventario
- Usuario 1 — N MovimientoInventario
- (y opcionalmente enlazar movimientos con Venta/Compra/Devolución)

Devoluciones

- Venta 1 — N Devolución
- Devolución 1 — N DetalleDevolución
- SKU 1 — N DetalleDevolución
- Devolución 1 — N MovimientoInventario

✓ Esta parte es la que tú dibujas en ER y te queda perfecto.

11.5 Paso 4 — Reglas de negocio que justificaron el modelo

Estas reglas “obligan” tablas/relaciones y te protegen del caos:

- **RB1 Inventario por movimientos:** el stock cambia solo mediante MovimientoInventario.
- **RB2 Venta tiene ítems:** Venta siempre tiene 1..N DetalleVenta.
- **RB3 Precio histórico:** DetalleVenta guarda precio aplicado y descuento (aunque el SKU cambie de precio luego).
- **RB4 Pedido puede morir:** Pedido puede quedar en borrador y no convertirse en venta.
- **RB5 Pagos parciales:** una venta puede tener varios pagos (saldo pendiente).
- **RB6 Devolución genera movimientos:** devolver productos genera entrada de inventario.

✓ Observación pro: Estas reglas son tu defensa cuando alguien dice “¿por qué no ponemos solo una tabla gigante?”

11.6 Paso 5 — Traducción a relacional (tablas que se desprenden)

Una vez fijo el ER, la traducción fue directa:

- category (1)
- product (N) → category_id

- sku (N) → product_id
- supplier
- purchase → supplier_id, created_by_id
- purchase_item → purchase_id, sku_id
- customer
- sale → customer_id (nullable), created_by_id
- sale_item → sale_id, sku_id
- payment → sale_id
- order → customer_id (nullable), created_by_id
- order_item → order_id, sku_id
- shipping → order_id (UNIQUE, opcional)
- sale.order_id (UNIQUE, opcional) • tabla puente order_sale (si el futuro permite 1 pedido → varias ventas)
- inventory_movement → sku_id, created_by_id, reference_type, reference_id
- return → sale_id, created_by_id
- return_item → return_id, sku_id
- role
- user → role_id (modelo simple)
- si es multi-rol → user_role

✓ Precaución: Las tablas intermedias (*_item) son normales: son tu forma correcta de modelar documentos con múltiples líneas.

11.7 Paso 6 — Normalización aplicada (lo importante del caso)

1FN

- Nada de listas en un campo (teléfonos separados por coma, etc.).

2FN

- Los detalles (sale_item, purchase_item, order_item) solo guardan cosas que dependen de su documento + SKU.

3FN

- No duplicar nombres de categoría o producto dentro de ventas.
- Venta guarda referencia y los detalles guardan precio histórico.

✓ Resultado real del caso: - Cambiar precios del catálogo no rompe historial porque el precio real está en `sale_item`.

11.8 Paso 7 — Modelo físico inicial (constraints + índices)

Constraints importantes

- `users.username` UNIQUE
- `roles.name` UNIQUE
- `categories.name` UNIQUE
- `skus.sku_code` UNIQUE
- `shipping.order_id` UNIQUE
- `sale.order_id` UNIQUE (si aplicas 1 venta por pedido)

Índices típicos

- FK indexados:
- `products(category_id)`
- `skus(product_id)`
- `purchases(supplier_id)`
- `sales(status)`
- `orders(status)`
- `inventory_movements(sku_id, created_at)`

✓ Observación: En V1, con estos índices ya camina sobrado con 5k SKUs y uso normal.

11.9 Paso 8 — Decisiones intencionales (lo que NO hicimos en V1)

Para que el proyecto sea entregable en 3 semanas, dejamos fuera: - Multi-bodega - Seriales/lotos - E-commerce real (carrito, checkout) - Facturación electrónica - Reglas avanzadas de precios (mayorista/minorista por cliente)

✓ Importante: No es "falta", es **scope control**.

11.10 Paso 9 — Preguntas que cambiarían el modelo si crece

Cuando el cliente crezca, estas preguntas te pueden obligar a rediseñar: - ¿Una sola bodega o varias? - ¿Se venden cosas por encargo sin stock? - ¿Se manejan lotes/seriales? - ¿Un pedido puede dividirse en varios envíos? - ¿Precio por cliente (mayorista)? - ¿Garantías?

🔄 Si aparece una de estas, el modelo se "ramifica" y no es el mismo V1.

11.11 Resultado final de este ejemplo

Un modelo que: - mantiene trazabilidad de inventario - permite ventas con pagos parciales - permite pedidos/cotizaciones por WhatsApp - corrige errores con devoluciones - no rompe historial - es lo suficientemente simple para V1, pero escalable con módulos futuros